Self-Evolution AI Training: The Teacher–Student Practice Loop

Briefing note for the ARC-AGI-3 FYP — applying self-evolution training to a small model. Compiled August 18, 2026.

1. The core idea

"Self-evolution" (also called recursive self-improvement, or RSI) describes a training setup in which a model gets better mainly by practicing against tasks it generates for itself, rather than by consuming a large fixed dataset of human-labeled examples. Instead of one static training set, the system runs a loop: generate a practice task, attempt it, check the outcome, learn from the result, and generate the next task — slightly harder than the last. Over many rounds the model's own output becomes its own curriculum.

The useful way to think about this is as two roles working together, even when they are played by a single network:

- **The teacher** (also called the *proposer*) invents practice problems.
- **The student** (also called the *solver*) attempts them and trains on the outcome.

This briefing summarizes how that loop actually works, why it is well suited to training a *small* model cheaply, and how it maps onto the ARC-AGI-3 FYP.

2. The loop, step by step

1. **Propose.** The teacher generates a new task — a puzzle, a coding problem, a grid environment, a sub-goal. It can do this by mutating a task that already exists, sampling from what it has learned so far, or searching for tasks that sit just beyond the student's current ability.
2. **Attempt.** The student tries to solve the task and produces an answer, a program, or a sequence of actions.
3. **Verify.** The attempt is checked automatically — by code execution, by a simulator, by matching a known invariant, or by a game/environment's own success signal. Critically, this step needs **no human labeling**: the check is mechanical.
4. **Reward and update.** Two reward signals are typically used together:
   - The **student** gets a reward for correctness (or efficiency) on the task it just attempted, and its policy is updated (usually with reinforcement learning) to make good attempts more likely.
   - The **teacher** gets a reward for proposing a task that was *useful* — one the student got partially right, or one that measurably improved the student's performance afterward — not simply one that was hard or novel for its own sake.
5. **Escalate and repeat.** As the student improves, tasks that used to be "just right" become too easy. The teacher's incentive automatically pushes it to keep proposing new tasks at the edge of what the student can currently do, so the curriculum tracks the student's growth without anyone hand-designing difficulty levels.

3. Two ways to set this up

Single model, dual role (self-play). One network is prompted or conditioned to act as proposer in one pass and solver in the next. Absolute Zero (Zhao et al., 2025) is the clearest example: a single model proposes coding tasks (in three flavors — predict the output of a program, infer the input that produced an output, or synthesize a program from examples) and then solves them, with every task checked by actually executing the code. The proposer is rewarded by a *learnability* score: tasks the solver sometimes succeeds on and sometimes doesn't are worth the most, because they carry information. Tasks that are always trivial or always impossible are worth little. With **zero human-curated training data**, this produced coding and math performance competitive with — and in places better than — models trained on tens of thousands of expert-written examples, and the benefit grew with model scale.

Separate teacher and student networks (asymmetric self-play). Here the teacher does not need to be able to solve the problems it proposes — its job is purely to find problems that move the student's ability forward. SOAR, described in "Teaching Models to Teach Themselves: Reasoning at the Edge of Learnability" (2026), works this way: a teacher model proposes problems for a student, and the teacher is rewarded by how much the student *improves* on genuinely hard held-out problems as a result. This is trained as a bilevel meta-reinforcement-learning problem — the teacher is optimizing its problem-generation policy against the student's learning curve, not against a fixed dataset. The paper's key finding is about what happens on datasets where the student starts at effectively 0% success: a well-sequenced curriculum of self-generated stepping stones can carry the student across that "edge of learnability," where training on the hard problems directly would produce no learning signal at all (nothing but zero rewards to learn from).

A related pattern, MetaEvolve (2026), trains the *skill of self-improvement itself* by showing a model a chain of its own past attempts (with their scores) and asking it to produce a better one, using execution-based reward in a competitive-coding

setting. This generalized well to domains the model wasn't explicitly trained on, suggesting "know how to improve your own last attempt" is a transferable skill rather than a task-specific trick.

4. What makes a practice task actually useful

Across all of these systems, three ingredients recur, and each is worth checking for when applying the idea elsewhere:

- **A cheap, automatic verifier.** Self-evolution only works if "did this attempt succeed?" can be answered mechanically — by running code, checking a physics/game simulator, or matching a formal invariant. Without this, the loop needs a human or a learned reward model in the loop, which reintroduces the cost and bias the method is trying to avoid.
- **A learnability-shaped reward for the teacher.** Rewarding the teacher for difficulty alone tends to produce tasks that are merely hard-looking or that collapse into a narrow, repetitive style (mode collapse / diversity collapse). Rewarding it for the student's *measured improvement*, or for a moderate, informative success rate, keeps the curriculum genuinely useful and diverse.
- **A feedback signal the student can actually learn from.** Binary pass/fail is the simplest and most robust; efficiency- or quality-weighted scores (e.g., runtime, action-efficiency, elegance) add a secondary training signal once basic correctness is solved.

5. Why this is a good fit for training a *small* model

Self-evolution training was originally explored to squeeze more capability out of large models without more human data, but several of its properties are arguably even more valuable for a small model with a constrained compute and data budget:

- **No large labeled dataset required.** A small model trained from scratch or fine-tuned on a narrow domain rarely has access to a big curated dataset in the first place. A self-generated curriculum sidesteps that entirely.
- **The curriculum matches the model's actual current ability**, automatically, because the teacher's reward is tied to the student's measured progress rather than a fixed difficulty schedule designed for a different (usually larger) model.
- **Verifiable-reward training is cheap and precise** compared to reward-model-based RLHF — there's no separate reward model to train (itself often a large model), which matters when the whole compute budget is small.
- **Evidence that meta-skills transfer**: gains from self-evolution training on one domain (e.g., code) have been shown to transfer to others (e.g., math, numerical optimization), suggesting a small model doesn't need self-play training data in every target domain — practicing on one verifiable domain can still improve general "propose a good next step / learn from my last attempt" ability.

The trade-off to be aware of: teacher rollouts, verification, and student rollouts all cost compute per training step, so self-evolution loops are usually *sample-efficient* rather than *compute-free* — the saving is in avoided data curation and reward-model training, not in raw FLOPs.

6. Mapping this onto the ARC-AGI-3 FYP

ARC-AGI-3's setup — an agent dropped into a novel grid environment with no stated rules or goals, needing to explore, build a world model, and act — is a natural fit for a self-evolution loop, and it connects directly to two of the candidate FYP directions already scoped (agent-building, and the more ambitious meta-learning/self-improving direction):

- **Teacher role → environment/curriculum generation.** Instead of (or alongside) hand-authoring more grid environments, a teacher process could procedurally mutate or compose existing ARC-AGI-3-style environments — new object counts, new rule combinations, new goal structures — specifically chosen to sit at the edge of what the current student agent can solve. This is the same move SOAR makes for reasoning problems, applied to environment design instead of problem statements.
- **Student role → the small agent being trained.** The student attempts each generated environment; the environment's own implicit feedback (state transitions, terminal/goal signals) plays the role of the "execution check" that code-based self-play uses — no human grading needed, which matters because ARC-AGI-3 explicitly withholds stated goals and rules.
- **Reward shaping.** The official ARC-AGI-3 efficiency metric (`min(1, human_baseline_actions / agent_actions)^2`) is a ready-made, verifiable per-level reward for the student. A learnability-style reward for the teacher — favoring environments the student sometimes solves and sometimes doesn't, or ones where student performance measurably improves after practice — would follow the SOAR/Absolute Zero recipe directly.
- **Relation to SOAR and evolutionary program synthesis**, already flagged in the FYP briefing as a candidate technique: this is exactly the self-improving evolutionary program synthesis pattern, and the newer "Teaching Models to Teach Themselves" work gives a more recent, better-documented recipe (bilevel meta-RL, edge-of-learnability targeting) that could be adapted or cited as prior art.
- **Why "small model" specifically matters here**: given the current state of the art on ARC-AGI-3 is extremely low (frontier systems around 0.5% aggregate, the best preview agent around 12.6% on public levels but collapsing to ~0.25% on the private set), brute-force scaling is clearly not the bottleneck — sample efficiency and generalizing from few self-generated examples is. That is precisely the regime self-evolution training is designed for, and it lines up with other small-model

precedents already noted in the briefing (Tiny Recursive Model at ~7M parameters, CompressARC at ~76K parameters).

7. Pitfalls to design around

- **Reward hacking / degenerate tasks.** A teacher optimized purely for "student improves" can learn to propose narrow, repetitive, or trivially-gameable tasks. Diversity or novelty terms, or capping how similar consecutive proposals can be, help offset this.
- **Diversity collapse.** Both Absolute Zero and SOAR-style work note that a naive learnability signal can converge on one easy pocket of task-space. Explicit incentives for coverage/novelty are usually needed alongside learnability.
- **Verifier gaps.** Any place where "success" can be satisfied without actually solving the intended problem (a bug in the checker, an under-specified goal condition) will be found and exploited by the student — the verifier needs to be as carefully designed as the reward itself.
- **Compute accounting.** Because every step needs a generation *and* a verification *and* a training update, self-evolution loops can be slower per wall-clock step than supervised fine-tuning on a fixed dataset, even though they need far less curated data. Budget for this when comparing against simpler baselines.

8. Key terms (quick glossary)

Term	Meaning
Recursive self-improvement (RSI)	A model uses its current ability to improve the process that produces its ability.
Teacher / proposer	The role generating practice tasks or curricula.
Student / solver	The role attempting tasks and being trained on the outcome.
Self-play	A single model (or matched pair) alternating between the teacher and student roles.
Learnability reward	A reward for the teacher based on how informative a task is for the student (neither trivial nor impossible), not on raw difficulty.
Edge of learnability	The curriculum boundary where a task set the student cannot yet solve (near-0% success) becomes reachable through intermediate practice tasks.
Verifiable reward	A reward computed mechanically (code execution, simulator, environment signal) rather than by a human or learned reward model.
Bilevel meta-RL	Training the teacher's proposal policy against the student's *learning progress*, rather than against a fixed objective.

9. Sources

- Zhao et al., "Absolute Zero: Reinforced Self-play Reasoning with Zero Data" — arXiv:2505.03335
- "Teaching Models to Teach Themselves: Reasoning at the Edge of Learnability" (SOAR) — arXiv:2601.18778
- "MetaEvolve" (self-evolution meta-skills via RL on evolution trajectories) — arXiv:2607.21971
- Lilian Weng, "Harness Engineering for Self-Improvement" — lilianweng.github.io
- ARC-AGI-3 scoring, state of the art, and small-model precedents (Tiny Recursive Model, CompressARC, SOAR) — see the companion project doc *ARC-AGI-3 Briefing*.